Milestone One

Caleb Hadley

Faculty of Computing & Data Sciences

CDSDX-799: Data Science Capstone

Joshua Von Korff

June 28, 2026

Problem Statement / Description

This paper develops a unified framework for financial crime detection utilizing account and transaction features. Applied across the IBM Transactions for AML, Paycheck Protection Program (PPP), and the European Credit Card (ECC) Fraud Detection datasets, the framework identifies common challenges like class imbalance, high false-positive rates, and overfitting by evaluating a number of supervised classification models. The presented solutions provide data scientists with strategies to combat financial crime in their organizations to save money, ensure regulatory compliance, and maintain customer confidence.

Conclusions

Polynomial and Interaction Terms (Breadth)

After undersampling the IBM dataset to a 10:1 ratio of negative to positive class to overcome class imbalance, there were 51,770 negative class instances and 5,177 positive class instances and 39 features. The baseline logistic regression model AP was 0.11 on the test set. I calculated VIF and found 34 features greater than five, indicating multicollinearity. I dropped duplicative features and engineered a ForeignExchangeTransaction feature instead of using the individual Payment/Receiving feature for each currency. After this, the AP on the remaining 20 features was 0.14, a small lift over baseline performance. Model performance improved when addressing multicollinearity but adding polynomial terms did not further increase the AP.

In the PPP dataset, resampling resulted in 950 negative and 95 positive class instances with 42 features. The preliminary model achieves 0.20 AP with 31 features containing VIF greater than five, indicating multicollinearity. After pruning features with high VIF, this resolved the multicollinearity and retained 16 features with 0.09 AP, a significant drop in performance.

This indicates that addressing multicollinearity could have removed information from the model, or the preliminary model could have been overfit.

The undersampling on the ECC dataset resulted in 4,920 negative and 492 positive instances with 30 features. I calculated AP using all 30 features as 0.84. This dataset's features were preprocessed to be PCA transformed, indicating noise and multicollinearity was already addressed. Next, I calculated VIF on all features, and found only minor multicollinearity. I removed the Amount feature with VIF of 12.12, retaining 29 features. After creating polynomial terms, the AP drops to 0.09. Although the baseline model performed well, adding polynomial terms decreased performance, due to the dimensionality, multicollinearity and overfitting.

Logistic Regression, Ridge and Lasso (Depth)

I tuned the hyperparameter C of logistic regression on the IBM dataset using a logarithmic grid from 0.001 to 1.0 and plotted the evaluation metrics against the C value, as shown in Figure 1. We can see the recall and f1 performance improve with a larger regularization penalty, approaching 0.86 recall and 0.88 f1. I recommend a C value less than 0.09, which maintains the strongest recall to help the model catch fraudulent transactions. Next, I used ridge and tuned the alpha parameter using a logarithmic grid from 0.001 to 1,000 to compare the logistic regression and ridge loss functions. In Figure 2, we can see that performance is equivalent, with more regularization increasing performance. Examining the ridge coefficients, 60 features comprising transaction amount summary statistics are almost completely shrunk to zero. This showcases that the features did not add additional information. Finally, using a lasso model we see the same features coefficients shrunk, in this case entirely to zero, while maintaining the same performance (see Figure 3). Many features did not add additional information to the model and their importance could be reduced. Finally, the logistic regression,

ridge, and lasso had strong performance, each approaching 0.89 macro f1 with consistent results across the negative and positive classes. The models identified the majority of fraudulent transactions without large numbers of false positives, while still remaining interpretable.

I performed the same hyperparameter tuning on a logistic regression on the PPP dataset. We can see in Figure 4 that regularization has a negative impact on the performance. Although the recall remains around 0.9, the precision decreases as regularization is added. Regularization could have constrained the model too much, penalizing complex patterns. However, overfitting is a risk with this dataset to sample size. I recommend balancing performance with variance reduction by choosing C equals 0.05. Next, I tuned the alpha hyperparameter on the ridge classifier. We see in Figure 5 that performance stays flat throughout the tuning, indicating the model performance is stable without tuning. Finally, using lasso we see a similar trend to logistic regression, with too much regularization drastically reducing model performance. A C value around 0.003 balances performance without overfitting.

The same logistic regression hyperparameter tuning on the ECC dataset showed that removing regularization constrained the model too much (see Figure 7). This makes sense in context of the dataset, as the features were already PCA transformed in preprocessing. As a result, there is not a high variance issue in this dataset and regularization is not necessary. I recommend utilizing the default C value for this dataset. Next, I confirmed that regularization is not needed by using a ridge model and performing parameter tuning of the alpha parameter. We can see in Figure 8 that the alpha curve is flat until the model collapses around 10 alpha, indicating that regularization does not have a positive effect.

Forward and Backward Selection, Principal Component Regression (PCR), Partial Least Squares Regression (PLSR) (Breadth)

In the IBM dataset, utilizing standardization and PCR with logistic regression resulted in 0.144 AP, while PLSR with a threshold of 0.5 for binary classification resulted in 0.143 AP. Examining the classification report, both models achieve f1 on the negative class of 0.95. The positive class performance is poor, with precision of 0.79 and recall of 0.07 on PLSR, and 0.80 precision and 0.08 recall on PCR. These models exhibit an issue with dimensionality reduction on datasets with large class imbalance. The models identify the components that result in the most variance in the dataset. As a result, rare patterns such as fraud can be discarded.

The PPP dataset PCR and PLSR models both result in 0.52 AP. We see a large lift over the original logistic regression performance. The PCA and PLSR successfully increases the performance of the logistic regression model significantly. Dimensionality reduction reduces the noise and the multicollinearity due to the transformation onto uncorrelated space.

For the ECC dataset, PCR achieved 0.86 AP on the test set, while PLSR achieved 0.62 AP. Using PCR we see a minor 0.01 increase in AP over the standard logistic regression. The PCA step could have reduced some noise in the dataset, helping the model to generalize better. Overall there was not much multicollinearity in the dataset due to the already PCA transformed features, so it is expected that this approach did not yield much change.

Logistic Regression and Feature Scaling (Breadth)

On the IBM dataset without scaling the logistic regression attains AP of 0.56 and lifts to 0.84 after scaling. The initial model had strong recall on the positive class, with 0.98 on the test set and 0.56 precision. The negative class precision was high with 0.93 but only 0.22 recall. Once scaling was added, the precision and recall are between 0.88-0.89 for both negative and positive classes. Examining the weights, the unscaled model had different scales for the amounts than the indicator features, resulting in unequal shrinkage.

The baseline logistic regression on the PPP dataset gets 0.41 AP. The recall was strong for both positive and negative class with 0.89 and the precision for the negative class was 0.99, but the positive class precision was only 0.45 which pulled down the AP performance. After scaling, we see a lift to 0.53 AP. The negative class f1-score moves up to 0.97, and the positive class precision lifts to 0.72 and recall sees a drop to 0.68 recall. Overall the model catches fraud more consistently, flagging less false positives. We see a lift in the performance with feature scaling. This could be due to large scaled features such as the amount, which end up with naturally smaller coefficients, causing the model to under-penalize them.

On the ECC dataset, a baseline logistic regression without feature scaling attains an AP of 0.85. In the classification report the negative class f1-score is 0.99 and the positive class has 0.95 precision and 0.89 recall. After applying feature scaling, there is a lift in AP to 0.86. The precision on positive class lifts to 0.96 and the recall to 0.89. Looking at the initial model, we see 9062 n_iter and only 26 on the scaled model. This shows that there were issues converging without scaling due to the mismatch between the amount and PCA transformed features.

Support Vector Machines, the Kernel Trick, and Regularization for Support Vector Machines (Breadth)

Using SVM classifier (SVC) on the IBM dataset with no parameter optimization or feature scaling yielded an AP of 0.50 . The positive class has poor performance with 0.0 recall. Next I perform feature scaling, which increases AP to 0.84 . Recall on positive class jumps from 0.0 to 0.88. Next I performed parameter tuning with stratified k fold cross validation. I tuned the C (0.001 to 1 on log scale), kernel using rbf and sigmoid, the coef0 parameter, and the class_weight (balanced and None). After 100 rounds, the best model utilized 0.12 C, no class weight, 0.75 coef0, and sigmoid kernel. The tuned model attains 0.82 AP on the test set.

In the PPP dataset, the model attains 0.79 AP without feature scaling. After scaling the AP increases to 0.77 . With parameter tuning it found the best parameters to be C equals 0.62 , Class weight none, coef0 equals 0.65, and kernel sigmoid which achieved 0.79 AP. These models did generalize to the test set, but with the limited number of samples (38 in the test set) it is possible the model actually overfit.

Repeating the same experiment on the ECC dataset, the SVC model gets 0.52 AP without feature scaling. We see lift to 0.96 AP with only feature scaling applied. With parameter tuning, we find the best model uses C equals 0.95, class weight balanced, coef0 equals 0.54, and kernel rbf. It gets 0.96 AP on the test set, matching the previous feature scaled model. Parameter tuning did not have an impact. In the context of this dataset, the regularization did not have an impact, and a linear kernel still performed well since the majority of features are already PCA transformed.

Decision Trees and Random Forests (Breadth)

The IBM dataset baseline decision tree classifier gets 0.39 AP. It has 0.96 f1 on the negative class and 0.60 precision and 0.59 recall on the positive class. The baseline random forest receives 0.40 AP, with 0.96 f1 on the negative class and 0.62 precision and 0.60 recall on the positive class. After parametering tuning the random forest model we get 0.50 AP. The positive class performance lifts to 0.86 recall at the cost of 0.56 precision. Of the two preliminary models, the decision tree performs better. It receives similar performance as the random forest while still being interpretable for stakeholders.

The PPP dataset baseline decision tree achieves 0.60 AP on the test set with 0.98 f1 on the negative class and 0.78 precision and 0.74 recall on the positive class. The baseline random forest matches the decision tree performance exactly. After performing the hyperparameter

tuning on the random forest, the AP drops to 0.40 the negative class f1 drops to 0.93, but the positive class precision is 0.44 and recall is 0.89. This model used max_depth of 2, min_samples_split 7, and n_estimators 139. The simplicity of the decision tree classifier worked best on this dataset, likely due to the PCA transformation applied to the majority of features in preprocessing.

On the ECC dataset, the baseline decision tree classifier gets 0.73 AP on the test set, while the random forest gets 0.87. Examining the classification report, the random forest has perfect f1-score on the negative class, and 0.99 precision and 0.87 recall on positive class. I performed parameter tuning using stratified five fold cross validation on the random forest, resulting in the parameters max_depth equals 3, min_samples_split equals 6, and n_estimators equals 27. Using these parameters the model gets 0.81 AP on the test set. This model maintains perfect f1 on the negative class, and 0.91 precision and 0.89 recall on the positive class.

Overfitting

I encountered overfitting due to extreme class imbalance and implemented undersampling to a 10:1 negative to positive class ratio to reduce the imbalance. Next, I utilized a train test split to validate model performance to ensure generalization to unseen data. Then, I performed five fold stratified cross validation on the training set to determine optimal parameters, using stratification to ensure folds contained both negative and positive class instances and cross validation for consistency with small fold sample sizes. In week one, I encountered overfitting due to the curse of dimensionality with a large count of features relative to the number of instances and calculated VIF on the features, pruning features which had high multicollinearity. In week two I used regularization techniques such as ridge and lasso to prevent overfitting by reducing dimensionality of the dataset and feature importance using coefficient

shrinkage and feature selection. In week three, I utilized dimensionality reduction techniques, which performed significantly better than the week one techniques for datasets like PPP. In week five, I addressed overfitting with SVM by hyperparameter tuning, resulting in lower values of the C hyperparameter for IBM and PPP datasets, which should encourage a wider margin and increased tolerance of misclassification. In week six, I reduced overfitting by constraining how the random forest depth could grow using parameter tuning of the max depth.

Metrics and Hyperparameter Tuning

All three datasets I analyzed included large class imbalances. As a result I selected the average precision (AP) score, recall, and f1 for model evaluation and selection. Additionally, I used the classification report and examined precision, recall and f1 for the models across each class to gain a deeper understanding of the tradeoffs between models. For hyperparameter tuning, I used Optuna for optimization and stratified five fold cross validation, which helped to address the small instance count in each dataset after performing undersampling by ensuring models generalized consistently rather than relying on a single train test split. Stratification was used to prevent folds from containing purely the majority class. To evaluate the impact of hyperparameter tuning, I compared performance of tuned models against baseline models.

Exploratory Data Analysis

In the EDA, I found that the features in the Credit Card dataset were scaled and PCA transformed to protect confidentiality of the publishing company's data. This informed why baseline models in this milestone performed strongly relative to other datasets. It also indicated why PCA and PLSR did not have measurable improvement over standard models. Across all three datasets, significant class imbalance informed my choice of undersampling, evaluation metrics, and focus on reduction of overfitting. Finally, in the EDA I found a large difference in

the scale of the transaction amount features in all three datasets relative to other features. Especially in the IBM dataset, all features were binary indicator features except the transaction amount. This informed the necessity of feature scaling. This was specifically impactful in the logistic regression model, where we saw failure to converge without scaling.

Expected vs. Unexpected Results

During evaluation of the SVM on my datasets I ran into issues due to kernel selection when attempting to use the polynomial kernel. Due to an explosion in training time, I had to limit my analysis across sigmoid, rbf, and linear kernels. I was also surprised to discover that simpler models such as logistic regression, ridge and lasso were able to achieve performance similar to the ensemble models like random forest. Additionally, the baseline performance of the ridge and lasso on scaled data outperformed the baseline random forest on the test set. I did not expect that the random forest would require more hyperparameter tuning to attain similar results. My final model recommendation as a result is the lasso, which performed well across each dataset while still being interpretable and requiring less hyperparameter tuning.

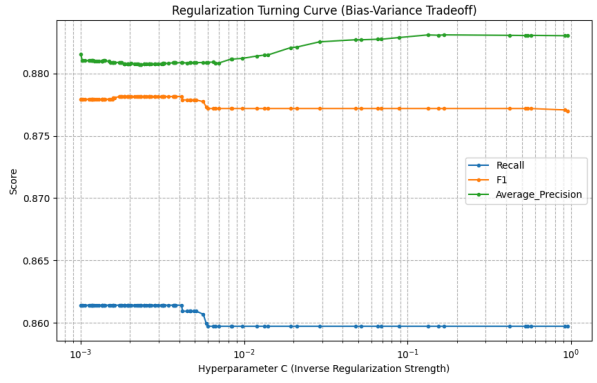

Figure 1. IBM Logistic Regression C Tuning

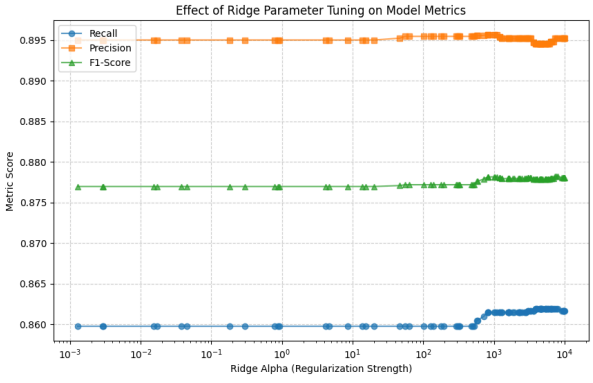

Figure 2. IBM Ridge Parameter Tuning

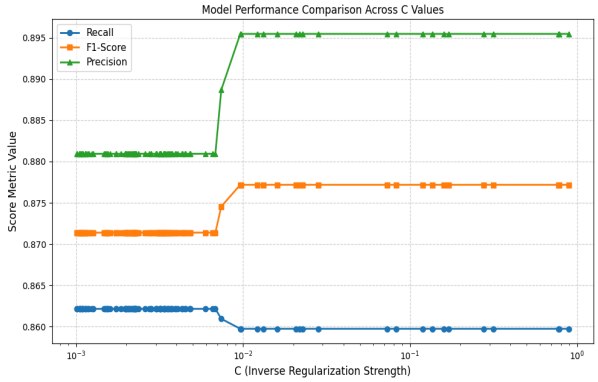

Figure 3. IBM Lasso Parameter Tuning

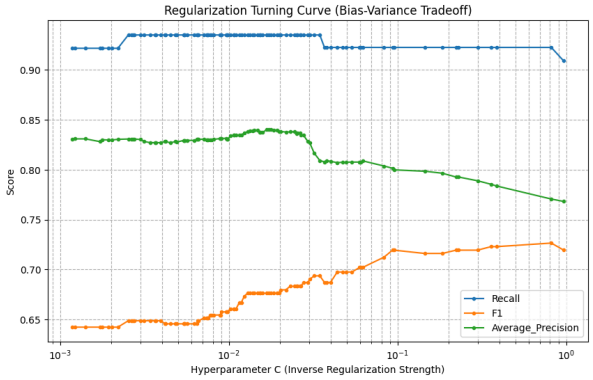

Figure 4. PPP Logistic Regression C Tuning

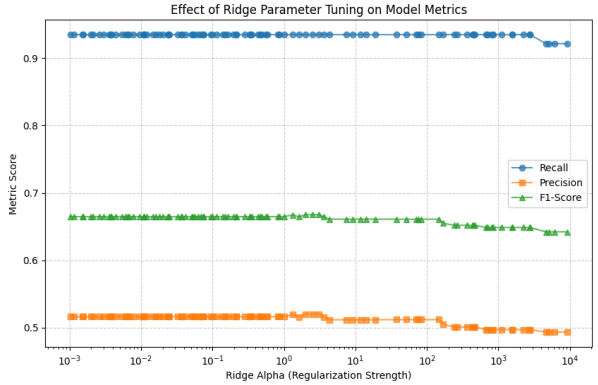

Figure 5. PPP Ridge Parameter Tuning

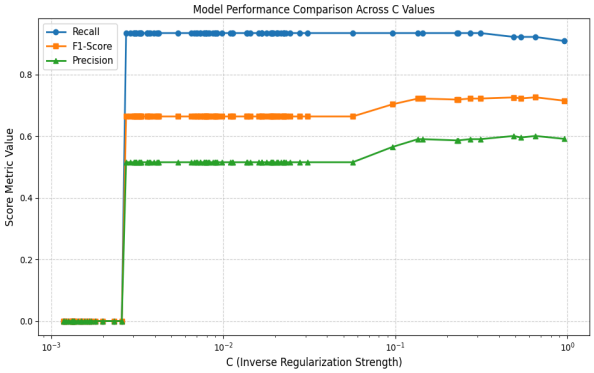

Figure 6. PPP Lasso Parameter Tuning

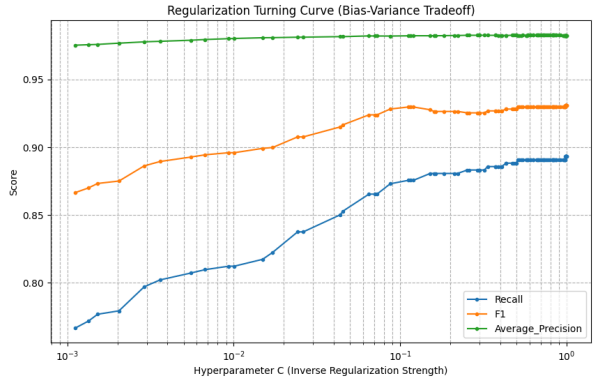

Figure 7. ECC Logistic Regression C Tuning

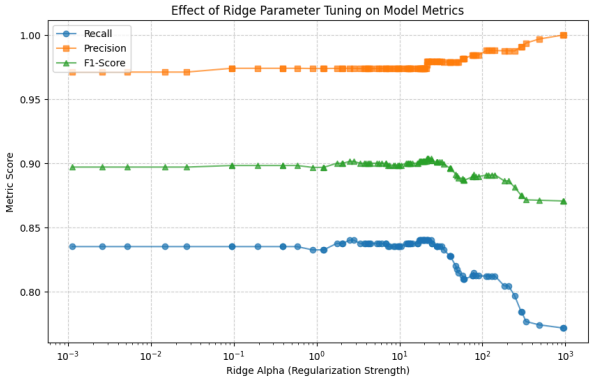

Figure 8. ECC Ridge Parameter Tuning

References

Altman, E. (2025, July 8). *IBM transactions for Anti Money Laundering (AML)*. Kaggle. https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml

Bukowski, D. (2022, May 25). *Covid PPP loan data with fraud examples*. Kaggle. https://www.kaggle.com/datasets/danb91/covid-ppp-loan-data-with-fraud-examples

Machine Learning Group - ULB. (2018, March 23). *Credit Card Fraud Detection*. Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

AI Appendix

I did not use AI for this submission.